

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: July 1, 2026
Semester: 1st Semester	Session: Spring 2026
Lab/Project/Assignment #: 11	CLOs to be covered: CLO4
Lab Title: Exception Handling	Teacher Name: Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO4	Implement error handling and debugging techniques to build robust, fault-tolerant programs.					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 11

Lab Goals / Objectives:

- Understand what exceptions are and why they occur
- Learn to use try-except blocks for error handling
- Handle multiple exception types
- Use the else and finally clauses
- Raise exceptions intentionally with the raise keyword
- Understand common built-in exception types
- Write robust programs that handle errors gracefully

Equipment Required:

- Computer System with Python 3.x installed
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

Theory:

1. What are Exceptions?

An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions. When an error occurs, Python raises an exception which can be caught and handled to prevent the program from crashing.

2. Common Types of Exceptions

Python has many built-in exception types:

- `ZeroDivisionError`: Raised when dividing by zero
- `ValueError`: Raised when an operation receives an argument with the right type but inappropriate value
- `TypeError`: Raised when an operation is applied to an object of inappropriate type
- `IndexError`: Raised when a sequence index is out of range
- `KeyError`: Raised when a dictionary key is not found
- `FileNotFoundError`: Raised when a file operation fails
- `NameError`: Raised when a variable is not defined

3. The try-except Block

The try-except block is used to catch and handle exceptions. Code that might raise an exception is placed in the try block, and the handling code is placed in the except block:

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print('Cannot divide by zero!')
```

4. Catching Multiple Exceptions

Multiple except blocks can be used to handle different types of exceptions. A single except can also catch multiple exceptions using a tuple:

```
try:
    x = int(input('Enter a number: '))
    result = 10 / x
except ValueError:
    print('Invalid input!')
except ZeroDivisionError:
    print('Cannot divide by zero!')
```

5. The else Clause

The else clause executes if no exception was raised in the try block. It is useful for code that should run only when the try block succeeds:

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print('Error!')
else:
    print('Result:', result)
```

6. The finally Clause

The finally clause always executes, regardless of whether an exception occurred. It is commonly used for cleanup operations like closing files:

```
try:
    f = open('file.txt')
except FileNotFoundError:
    print('File not found!')
finally:
    print('Cleanup complete.')
```

7. The raise Keyword

The raise keyword allows you to intentionally raise an exception. This is useful for enforcing constraints or signaling errors in your own code:

```
def check_age(age):  
    if age < 0:  
        raise ValueError('Age cannot be negative!')
```

8. Exception Hierarchy

All built-in exceptions in Python inherit from the `BaseException` class. The most common base class for user-defined exceptions is `Exception`. Catching `Exception` will catch most common errors.

9. Best Practices

- Catch specific exceptions rather than the general `Exception` class
- Use `finally` for cleanup operations that must always run
- Do not use exceptions for normal flow control
- Provide meaningful error messages to help debugging
- Log exceptions for troubleshooting in production code

10. Sets Review

Sets are unordered collections of unique elements. They support mathematical operations like union, intersection, and difference. The `issubset()` and `issuperset()` methods check set relationships.

Lab Work:

Task 1: Basic try-except

Write a program that takes a number as input inside a try block and catches any input error with a general except.

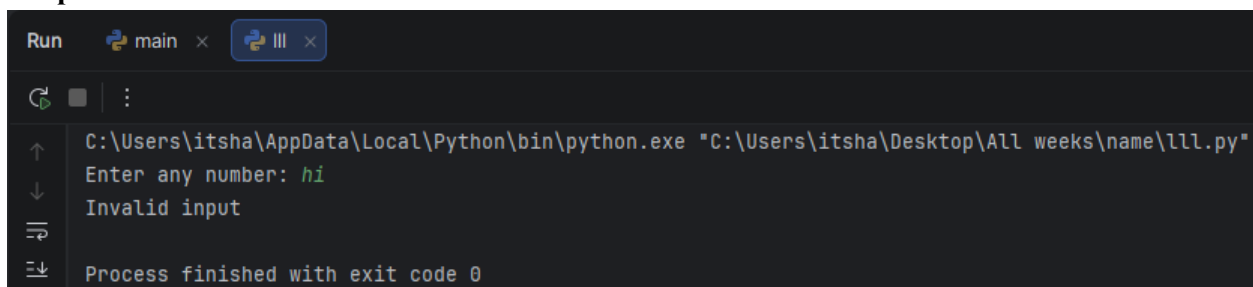
Code Summary:

This task shows the simplest form of exception handling: risky code (converting input to an integer) is placed in a try block, and a bare except catches any error that occurs, preventing a crash.

Key Code Lines:

```
# Task 1: Basic try-except
try:
    number = int(input("Enter any number: "))
    print("Hello World")
except:
    print("Invalid input")
```

Output:



```
Run  main x  III x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\111.py"
Enter any number: hi
Invalid input
Process finished with exit code 0
```

Task 2: Catching the Exception Object

Write a program that catches an input error and prints the actual exception message using 'except Exception as e'.

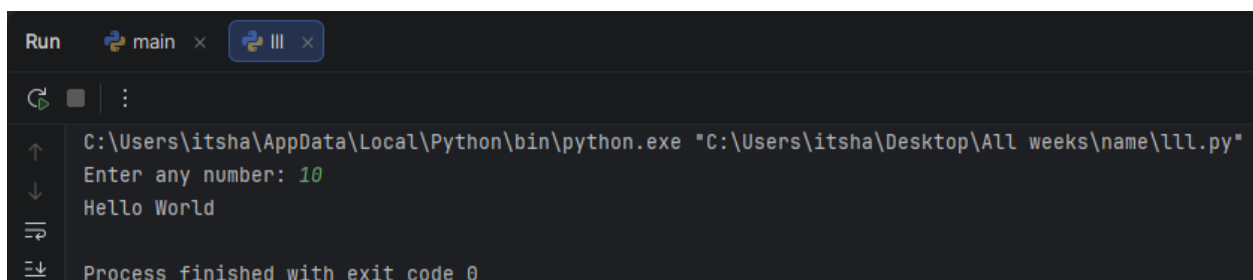
Code Summary:

This task demonstrates capturing the exception object itself with 'as e', which lets the program display Python's own error description instead of a generic message.

Key Code Lines:

```
# Task 2: Catching the Exception Object
try:
    number = int(input("Enter any number: "))
    print("Hello World")
except Exception as e:
    print(e)
```

Output:



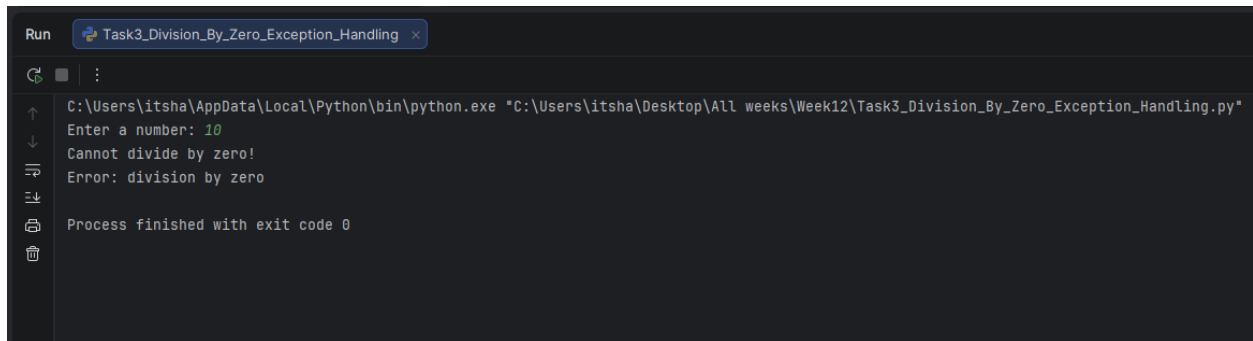
```
Run  main x  III x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\111.py"
Enter any number: 10
Hello World
Process finished with exit code 0
```

Task 3: Division by Zero Handling

CODE:

```
num = int(input("Enter a number: "))
try:
    ans = num / 0
    print("Result:", ans)
except ZeroDivisionError as e:
    print("Cannot divide by zero!")
    print("Error:", e)
```

Output:

A screenshot of a Python IDE window titled "Task3_Division_By_Zero_Exception_Handling". The window shows the execution of a Python script. The command prompt displays the file path "C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week12\Task3_Division_By_Zero_Exception_Handling.py". The user input "Enter a number: 10" is shown, followed by the program output "Cannot divide by zero!" and "Error: division by zero". The window concludes with "Process finished with exit code 0".

```
Run Task3_Division_By_Zero_Exception_Handling x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week12\Task3_Division_By_Zero_Exception_Handling.py"
Enter a number: 10
Cannot divide by zero!
Error: division by zero
Process finished with exit code 0
```

Conclusions:

- Successfully understood what exceptions are and why they occur
- Learned to use try-except blocks for graceful error handling
- Implemented handling for multiple exception types
- Understood the purpose and use of else and finally clauses
- Learned to raise exceptions intentionally with the raise keyword
- Reviewed set operations including subset and superset checking
- Wrote robust programs that handle errors without crashing